

04 — O que já existe

Inventário do que está construído, sem embelezar. Serve para saber o que se aproveita e o que tem de nascer.

No Piloto — é uma demonstração, e é preciso dizê-lo

O Piloto tem o **aspecto** da facturação electrónica sem nada por baixo. Três exemplos, todos verificados no código:

O QR Code não é um QR Code. `assets/js/fatura-doc.js`, função `pseudoQR()` : desenha um quadrado de 21×21 com os módulos escolhidos por um `hash` do número do documento, e acrescenta os três quadrados dos cantos para parecer verdadeiro. Nenhum leitor lê aquilo. E o QR da AGT tem de ser versão 4 (33×33), correcção M, e conter uma URL de consulta.

O código de validação é inventado. `assets/js/comercial.js` :

```
function hash32(str) { let h = 5381; ... } // djb2
function codigoValidacao(v) { return hash32(numero + total + cliente + data) ... }
```

Um resumo de quatro caracteres, sem chave, sem encadeamento, sem relação com a AGT. O DP 71/25 (art. 19.º) diz que o código de autenticação é **definido pela Administração Tributária**.

O SAF-T exporta um CSV. Em `empresa.html`, o botão diz «Exportar SAF-T (demo)» e produz linhas separadas por vírgulas. O SAF-T é XML validado contra um XSD.

O que o Piloto tem de aproveitável: a configuração de facturação — modo (electrónica / SAF-T / ambos), software, versão, nº de certificado, ambiente (testes/produção), série e periodicidade. Os campos estão certos; é a implementação que não existe.

Na Produção — a base está lá, a comunicação não

O modelo `Venda` já guarda:

Campo	Serve para
<code>numero</code> (<code>FT 2026/0001</code>)	Numeração sequencial — atribuída só na emissão
<code>tipo_doc</code> (FT, FR, NC, ND...)	Tipo de documento
<code>data</code> , <code>emitido_em</code>	Datas
<code>cliente_id</code> + <code>cliente_nome</code>	Cliente, com cópia do nome para consumidor final
<code>iva_perc</code> , <code>subtotal</code> , <code>iva</code> , <code>total</code>	Valores
<code>codigo_validacao</code>	Existe — mas com o mesmo problema do Piloto
<code>estado</code>	rascunho → emitida
<code>lancamento_id</code>	A ligação à contabilidade
<code>linhas</code>	Artigo, quantidade, preço, ordem

E há mais coisas que ajudam e que não estavam previstas neste plano:

- **A consulta de NIF à AGT** já está construída (`services/nif.py`), com

autenticação Basic corrigida — a mesma família de serviços.

- **O gerador de QR PNG com logótipo** já existe, feito para o segundo factor

(`auth/totp.py`), com correcção de erros alta e a marca ao centro medida para não partir a leitura. Os parâmetros da AGT são outros, o mecanismo é o mesmo.

- **O plano de contas PGC-AR** completo, que é o `GeneralLedgerAccounts` do

SAF-T.

- **A paginação e o histórico** de vendas, que a listagem de facturas

comunicadas vai reutilizar.

O que falta, resumido

Peça	Estado	Para quê
Tabela de impostos normalizada	✗	SAF-T e API
Séries como entidade	✗	SAF-T e API
Encadeamento por <code>hash</code>	✗	SAF-T (e é o mais sensível)
Código EAC da empresa	✗	Ambos
Motivo de isenção por linha	✗	DP 71/25 art. 10.º f
Hora e local da operação	✗	DP 71/25 art. 10.º g
País do cliente	✗	API
Unidade de medida por linha	⚠ parcial	API
Gerador de XML SAF-T	✗	SAF-T
Validação contra o XSD	✗	SAF-T
Chaves da AGT por empresa	✗	API
Assinaturas JWS	✗	API
Fila e estado de submissão	✗	API
QR Code no modelo da AGT	✗	Documento impresso
Certificação do software	✗	Fora do código — é um processo com a AGT

Uma nota sobre a certificação

O `softwareValidationNumber` que entra na assinatura e na factura impressa **vem da AGT**, depois de o software ser validado. Isso não se programa: é um processo com a Administração, e o [Decreto Executivo n.º 74/19](#) diz o que o software tem de cumprir para o obter.

Vale a pena ler esse documento cedo, porque há requisitos que são mais fáceis de cumprir enquanto se constrói do que de remendar depois — nomeadamente os que dizem respeito a inviolabilidade dos registos e a registo de alterações.